Measuring AI code assistants and agents **Read whitepaper**

AI productivity debate

Researchers and practitioners weigh in on AI's current and future impact.

Justin Reock
Deputy CTO

This post was ***originally published*** *in Engineering Enablement, DX's newsletter dedicated to sharing research and perspectives on developer productivity.* ***Subscribe*** *to be notified when we publish new issues.*

There are several assumptions about AI circulating in executive teams and boardrooms that are shaping strategy today. For example, some leaders believe AI will ultimately mean fewer developers, or at least fewer junior engineers. Some believe AI adoption must be mandated from the top in order to take hold.

These beliefs are rarely black and white. So at DX Annual, we convened a panel of senior engineering and research leaders to debate these questions in the open. Our aim was to surface the nuance behind these assumptions, give our audience exposure to contrasting viewpoints, and equip them to articulate their own position with greater clarity and confidence.

The panel:

- **Rafe Colburn**, Chief Product and Technology Officer at Etsy, where he's led engineering for 14 years through multiple waves of technology change
- **Jesse Adametz**, Sr. Director of Engineering, Platform Engineering at Twilio, overseeing developer tooling and productivity for thousands of engineers
- **Eirini Kalliamvakou**, Research Advisor at GitHub, whose work on developer productivity measurement has shaped how the industry thinks about engineering effectiveness
- **Collin Green**, Senior Staff UX Researcher at Google, where he studies developer workflows and the factors that drive or block productivity
- **Brian Houck**, coauthor of the SPACE framework (and now **Distinguished Researcher** at DX), whose research on AI's impact on engineering spans many thousands of developers and managers

Each panelist was read a series of statements and asked for their opinion. Below is a recap of each statement and the panelists' reactions. **You can watch the full session here.**

Will AI mean fewer engineers?

Statement: An AI-first SDLC means fewer engineers.

Rafe: Thumbs down. A job is a bundle of tasks, and AI is a clean substitute for some of them. People might not be doing those things anymore. But the demand for software isn't going away, it's going up. The price of building software is going to go down. So I don't think fewer engineers is a likely scenario in the near future. I bet against it.

Brian: I agree, it's not about fewer engineers, it's about doing more. But one nuance: in the future, what do we even define as a software engineer? The bundle of tasks we think of today that make up a software engineer may change. Writing code is a core part of the identity. The people who do that specific thing may go down, but the total number of makers and builders won't.

Collin: You'll probably be able to do the same amount of stuff with fewer people, but that doesn't mean demand will go down. Small companies that want to accomplish something will be able to get over the barrier to entry faster. It may actually increase demand for engineering skills.

Is AI accelerating technical debt?

Statement: AI is currently creating technical debt faster than it is helping us refactor it.

Eirini: Because you said "currently," yes. Things with code generation are moving at a pace where the rest of the system hasn't evolved enough to match it. We're also seeing the anticipation of technical debt throttling how much developers use agents. They're trying to balance the risk of something with the velocity of something, and it's mental math that has to happen in the moment. There are ways to manage against the accumulation, like event-driven or scheduled agents that do maintenance and hygiene for codebases. It's not a perfect solution, but it's working for now. The accumulation of technical debt is certainly a big issue.

Jesse: It's too definitive of a statement. We look at our engineers in quartiles, and our highest-performing engineers are also our highest-performing AI engineers. AI is an amplifier. If you've got high-performing engineers, they weren't putting garbage into the system before and they're still not. It's garbage in, garbage out.

Rafe: I've worked at Etsy for 14 years. Proportionally, are we creating more tech debt now than five or ten years ago? No. The old code is still there. We're creating a greater volume of code, but I'm not sure a greater proportion of it is tech debt. Automated code reviews are helping, and you can ask a coding agent to look at patterns in the code. The greater volume probably brings more tech debt in absolute terms, but I don't think it's an accelerator for tech debt at a greater rate than it's an accelerator for anything else.

Brian: Disagree with the others. Organizations are optimizing for PR velocity, not cleanliness. And cognitive debt, understanding our systems less as AI writes more of the code, is a form of technical debt that's growing. In the short term, tech debt is going up relative to PR velocity.

Collin: Technical debt is a business decision, not strictly an engineering one. If businesses have the same tolerance for risk and the same market pressures, they'll incur the same total amount. But viewing agentic development as low-cost increases the risk of imprudent decisions.

Eirini: The choice of what even are you building becomes so important.

Will most code be AI-generated within five years?

Statement: In five years, more than 50% of code in most organizations will be written by AI.

Collin: If we're talking about new code, yes, maybe even sooner than five years. But it's worth asking whether that's code that would have been written by humans otherwise, or just a larger total volume. Also worth considering how much of it will be rapidly rewritten.

Jesse: I interpreted the statement differently. If the claim is 50% of all code in a codebase, that's wild. Too many companies have too much legacy software, and there's no advantage to rewriting all of it.

Rafe: Agreed it has to be new code for the statement to hold.

Is the future of engineering about managing agents?

Statement: The future of software engineering is more about managing agents rather than about writing code.

Brian: In an idealized world, yes. But humans aren't great at multitasking. Microsoft's internal research shows that even experienced developers under time pressure revert to sequential, single-threaded agentic workflows. People with prior management experience (delegation, context-switching) will be more successful, but it's a different skillset that we're not hiring for yet.

Rafe: Thumbs up, loosely. Even using Claude Code today, you're already managing multiple agents, it's just abstracted away. The hobby-project vision of orchestrating 15 agents simultaneously isn't the future. AI will help manage that complexity. But will a large chunk of your work be mediated by at least one agent? Yes. Agent orchestration is not a full-time human job.

Jesse: As a director, my brain naturally goes to managing agents. But the identity of a software engineer matters. Not everyone wants to be a manager, and not everyone is good at it. A blanket statement that everyone will manage agents is too much.

Eirini: I interpreted "managing" as the real engineering work of setting agents up for success: defining intent, setting constraints and guardrails, providing context, and then verifying output. That's real engineering at a different level of abstraction. If that's what the statement means, it's a strong thumbs up.

Collin: Micromanaging agents is a failure mode. That leads to task-switching bottlenecks.

Do leaders need to mandate AI adoption?

Statement: Leaders need to mandate AI usage to make sure adoption is moving along.

Jesse: Not what we've seen. Twilio did light enablement (install defaults, basic guidance), and adoption took off on its own. No top-down mandate needed.

Rafe: People outside of engineering orgs often push for mandates because they're afraid of falling behind. But mandates lead to shallow adoption and "tokenmaxxing." AI adoption is inevitable, so why mandate something inevitable? If someone had said a year or two ago that every single talk at the DX conference would be about AI, people would have said no way—but is there anything else to talk about? Telling software engineers AI is going to be part of their job just feels like you're fighting last year's battle this year. Leaders should focus on removing friction and making it easy to create value, not pushing people.

Brian: I'm running a study of ~600 engineers and managers. The biggest disagreement: the majority of engineering managers think AI usage is a reasonable individual performance metric. Engineers disagree. That's a myth to dispel. Activity metrics are useful for understanding patterns, but should not be used as direct performance measures.

Jesse: If organizations aren't crystal clear about how AI will be measured, people will fill the gap with their biggest anxieties. Twilio isn't putting AI usage in career frameworks anytime soon, but the reality is you're compared to your peers.

Rafe: If you went to a job interview today and said "I don't use AI coding tools," it would be like saying you're a programmer who refuses to use a text editor. It would just sound strange. Adoption is inevitable without a mandate.

Is code review now the bottleneck?

Statement: The bottleneck of software delivery is no longer writing code, it's now reviewing code.

Collin: As Brian noted earlier in the day, developers only spend about 14% of their time writing code, so code was never really the bottleneck. The real bottlenecks are decision-making, prioritization, product design, support, and operations.

Jesse: AI is an amplifier. Whatever your organization's bottleneck used to be, it still is. For Twilio, deep work has been an issue for a long time and still is. The conversation has shifted to "agent experience," but the fix is the same: improve developer experience. Meetings, coordination tax, decision-making, these have always been the problems.

Brian: Code review is still a bottleneck, and it's increasing as more code is produced. But at Microsoft, some features take two or more years from planning to customer delivery. Going from two days to three days on review isn't the long pole. Planning and prioritization are.

Rafe: The perceived bottleneck of code review is what's interesting. When writing a PR took two days, a review delay was annoying but tolerable. When writing takes 10 minutes, the human process of finding a reviewer and waiting feels unbearable. The interruptions of human processes, unless they provide clear value, are grating.

Jesse: Twilio's data showed their highest-quartile AI developers took a ~14-point hit in perceived code review turnaround, but their actual median merge time decreased by multiple hours. It's pure perception.

Is risk the only thing holding back adoption?

Statement: The only thing holding engineers back on AI adoption is unnecessary worry about risk.

Eirini: Engineers feel deeply accountable for what ships. When something is high-risk or hard to undo, they pull back on how much they leverage agentic velocity. That's not irrational, it's human behavior that's hard to get over.

Brian: Risk is a big reason, but not the only one. Over a fifth of developers in our research cited concern about introducing defects and vulnerabilities as a top barrier.

Jesse: I question whether AI is actually increasing risk, since engineers were already introducing bugs before AI.

Rafe: The risk landscape has completely shifted. If we had a butter knife and now you have a chainsaw, the risks are different. The risks that you won't be able to cut down a tree have gone way, way down. But the risks that you might really make a mess are a lot higher. We have people using these tools who don't really know how a chainsaw is different than a butter knife — "Look, someone just gave me something, cool." And on the other hand, engineers who are like, "I don't want to use the chainsaw unless I really understand everything about how it works."

Collin: Beyond risk, there are basic enablement problems: helping people choose tools, configure them, and get started. Those aren't AI-specific, they're just adoption problems.

Are AI adoption problems really culture problems?

Statement: Most AI adoption problems are really culture and management problems, not tooling problems.

Rafe: Culture and management problems are real, but sometimes the next blocker is a tooling problem, or a process problem. Process is a form of tooling. At Etsy, with a 20-year-old codebase and deeply tenured engineers, stated and unstated processes create friction. It's all of the above. But if you're a leader and you're not giving people time to learn, you are doing it wrong. I used to ask engineers, "Are you working as fast as you can?" "I don't know." "Are you learning anything from what you're doing?" "No." Then you're working as fast as you can. You're not taking any time for that. Making space for learning solves 90% of these problems.

Collin: Most problems are human problems. Tool impediments exist, but the bigger barriers are incentives, anxiety, learning, and space to learn. Leaders who give people time to learn demonstrate they understand how the world actually works.

Eirini: The transformation is massive. Bottlenecks have shifted, processes aren't working as expected, metrics no longer mean the same thing, and people are having an identity crisis. It can't be just a tooling problem. Culture, motivation, and infrastructure all need to adjust.

Jesse: There's a compounding effect. For people who haven't started, it's getting harder to start. I told my VP a couple weeks ago, "I have to quit to be able to keep up with the industry." That's the day job. So if folks haven't gotten started, I can only imagine how much more daunting it's getting.

Brian: Everything is moving so fast that what you learned yesterday is irrelevant tomorrow. Long-term burnout and fear of falling behind are basic human challenges. No matter how complex the technology, it's always a human problem at the end of the day.

Eirini: Teams that did two-week group learning sprints (everyone uses agents for everything, no exceptions) saw much better results in adoption, engagement, and outcomes than teams where individuals learned on their own.

—

There's a common throughline in these discussions: AI is reshaping the task mix of software work, but not eliminating the need for engineers. Additionally, the biggest risks and opportunities sit in how leaders design roles, measure impact, and consider the full PDLC, not in whether AI can generate more code.

To go deeper, you can ***watch the full session here*** *and explore the rest of the DX Annual recordings on the* ***Sessions page.***

LAST UPDATED
May 20, 2026

GET STARTED

Want to explore more?

See the DX platform in action.

Get a demo

DX uses cookies to enhance your browsing experience, serve personalized ads or content, and analyze traffic. By clicking "Accept", you consent to our use of cookies.

Accept

...served.